# Hibernate & JPA Interview Handbook

## 1. Hibernate Overview

Hibernate is an ORM (Object Relational Mapping) framework that maps Java objects to database tables. It eliminates most JDBC boilerplate code.

## 2. Hibernate Architecture

Configuration → SessionFactory → Session → Transaction → Database.

**Configuration**: Reads hibernate.cfg.xml and mappings.
**SessionFactory**: Heavyweight, one per database.
**Session**: Lightweight, not thread-safe, contains Level 1 Cache.
**Transaction**: Ensures ACID properties.

## 3. Entity Lifecycle

**Transient**: New object, not managed.
**Persistent**: Attached to Session.
**Detached**: Session closed, object still exists.
**Removed**: Marked for deletion.

## 4. get(), load(), getReference()

**get()**: Executes SQL immediately, returns null if not found.
**load()**: Deprecated in Hibernate 6, returns proxy, throws ObjectNotFoundException when initialized if record doesn't exist.
**getReference()**: JPA equivalent, returns proxy and throws EntityNotFoundException when initialized.

## 5. Level 1 Cache

Enabled by default. One cache per Session. Stores managed entities by primary key. Same entity + same primary key + same Session returns object from cache. If the first lookup returns null, Hibernate does not cache null and queries the database again.

## 6. Level 2 Cache

Shared across Sessions. Must be explicitly configured (e.g. Ehcache). Improves performance across multiple Sessions.

## 7. Dirty Checking

Hibernate automatically detects changes made to persistent entities and generates UPDATE statements during flush/commit.

## 8. flush(), clear(), evict()

flush(): Synchronizes persistence context with database.
clear(): Removes all managed entities from Session.
evict(): Removes a single entity from the Session cache.

## 9. save(), persist(), update(), merge(), saveOrUpdate()

save(): Returns generated identifier.
persist(): JPA standard; doesn't return id.
update(): Reattaches detached entity.
merge(): Copies detached state into managed entity.
saveOrUpdate(): Saves new entity or updates existing one.

## 10. Fetch Types

EAGER loads immediately. LAZY loads when accessed (typically using proxies). Fetch types are mainly associated with entity relationships.

## 11. Cascade Types

PERSIST, MERGE, REMOVE, REFRESH, DETACH and ALL propagate operations to associated entities.

## 12. Common Annotations

@Entity, @Table, @Id, @GeneratedValue, @Column, @Transient, @OneToOne, @OneToMany, @ManyToOne, @ManyToMany, @JoinColumn.

## 13. JPA vs Hibernate

JPA is a specification. Hibernate is one of its implementations and provides additional APIs and features.

## 14. Best Practices

Use one SessionFactory per database. Keep Sessions short-lived. Prefer LAZY relationships where appropriate. Always use transactions. Avoid N+1 query problems by using fetch joins or entity graphs when appropriate.

## 15. Frequently Asked Interview Questions

• Difference between Session and SessionFactory?
• Difference between get() and load()?
• What is a proxy object?
• What is dirty checking?
• Explain Level 1 and Level 2 Cache.
• Difference between update() and merge()?
• Difference between persist() and save()?
• What happens on flush()?
• Explain entity states.
• What is the Persistence Context?